সফটওয়্যার ইঞ্জিনিয়ারিং: ধারণা, প্যারাডাইম ও SDLC ধাপ

SE কী · ভালো সফটওয়্যারের গুণ · প্রসেস ও প্যাটার্ন · ৭টি মডেল · ৬টি ধাপ — ১২ পাতায় পরীক্ষার প্রস্তুতি



↑ পুরো টপিক এক পাতায়: লাল = বেশি আসে, সবুজ = প্রয়োগ

সিলেবাসের যে অংশ

"Software Engineering and Information System Design — Concepts of **Software Engineering**, **Software Engineering paradigms**, Different phases of **software System Development**, Different types of information, qualities of information." মেট্রিক, ঝুঁকির সংখ্যা ও শিডিউলিং আছে প্রজেক্ট ম্যানেজমেন্ট লেসনে।

হাই-ইল্ড তালিকা

① NATO, গার্মিশ ১৯৬৮
② IEEE: পদ্ধতিগত + পরিমাপযোগ্য
③ স্পাইরাল = বোয়েম, ঝুঁকি-চালিত
④ V-মডেল: প্রতিটি ধাপের সাথে টেস্ট
⑤ SRS = রিকোয়ারমেন্টস ফেজের আউটপুট
⑥ বাগ ঠিক করার খরচ: ×১ → ×১০০

সংকেত: **হলুদ** = মূল শব্দ · **লাল বৃত্ত** = মুখস্থ রাখার তথ্য · **বেগুনি বক্স** = মনে রাখার কৌশল · **লাল বক্স** = পরীক্ষার ফাঁদ · **সবুজ** = সমাধানকৃত উদাহরণ

সূচিপত্র

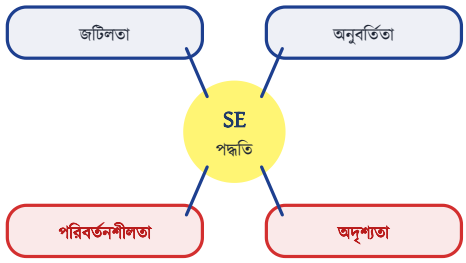
✓ SE কী · প্রসেস কেন ..... পৃ. ২
✓ ভালো সফটওয়্যারের গুণ ..... পৃ. ২
✓ প্রসেস · ফ্রেমওয়ার্ক · অনুশীলন ..... পৃ. ৩
✓ ১৪টি প্যাটার্ন · তথ্য ..... পৃ. ৪
✓ মডেল I–III ..... পৃ. ৫–৭
✓ ৭ মডেলের তুলনা ..... পৃ. ৮
✓ SDLC ধাপ ... পৃ. ৯ · ধাপ মানচিত্র ... পৃ. ১০
✓ রাতের পাতা পৃ. ১১ · অনুশীলন পৃ. ১২

১ সফটওয়্যার ইঞ্জিনিয়ারিং কী?

সংজ্ঞা: সফটওয়্যার ইঞ্জিনিয়ারিং (SE) হলো ইঞ্জিনিয়ারিং পদ্ধতি **পদ্ধতিগত, শৃঙ্খলাবদ্ধ ও পরিমাপযোগ্য**ভাবে সফটওয়্যারে প্রয়োগ করা — কীভাবে তা নির্দিষ্ট, ডিজাইন, তৈরি, যাচাই, ডিপ্লয়, রক্ষণাবেক্ষণ ও উন্নয়ন করা হয়। শব্দটি প্রথম ব্যবহৃত হয় **NATO, গার্মিশ ১৯৬৮** সম্মেলনে, যেখানেই **সফটওয়্যার ক্রাইসিস** নামটি এসেছে।

প্রসেস কেন অপরিহার্য

সফটওয়্যারের বৈশিষ্ট্য	কেন প্রসেস বাধ্যতামূলক
জটিলতা	কোনো physical সীমা এর ওপর নেই
অনুবর্তিতা	চুক্তি, মান ও আইন মানতে হয়
পরিবর্তনশীলতা	তৈরির সময়ও চলতে থাকে
অদৃশ্যতা	অগ্রগতি শুধু artifact দেখে বোঝা যায়



চারটি বৈশিষ্ট্য, যা সফটওয়্যারকে অন্য ইঞ্জিনিয়ারিং বস্তু থেকে আলাদা করে

মনে রাখার কৌশল: **জ-অ-পে-অ** — **জ**টিলতা, **অ**নুবর্তিতা, **পে**রিবর্তনশীলতা, **অ**দৃশ্যতা। "কোনটি সফটওয়্যারের বৈশিষ্ট্য **নয়**?" — উত্তর **নির্দিষ্টতা**, **ক্ষয়** বা **ক্লান্তি**।

২ ভালো সফটওয়্যারের বৈশিষ্ট্য (গুণাবলী)

গুণ	ঠিক কী বোঝায়	মূল শব্দ
সঠিকতা	SRS-এ যা চাওয়া হয়েছে তাই দেয়	"স্পেক মেনে চলে"
নির্ভরযোগ্যতা	প্রত্যাশিত অবস্থায় ব্যর্থ হয় না	MTBF
দক্ষতা	CPU, মেমরি ও I/O সর্বোত্তম ব্যবহার করে	সময় ও মেমরি
রক্ষণাবেক্ষণযোগ্যতা	সংশোধন, পরিবর্তন ও উন্নয়নের প্রচেষ্টা	"≈ ৭০% খরচ"
পোর্টেবিলিটি	অল্প পরিবর্তনে অন্য পরিবেশে চলে	bytecode
ব্যবহারযোগ্যতা	শিখতে সহজ, ভুল সহিষ্ণু ইন্টারফেস	user-friendly
নিরাপত্তা (integrity)	অনুমতিবিহীন প্রবেশ বন্ধ	গোপনীয়তা
নমনীয়তা	ছোট স্থানীয় বদলে নতুন ফিচার	modularity
পরীক্ষাযোগ্যতা	পুনরাবৃত্ত টেস্টে যাচাইযোগ্য	verifiability

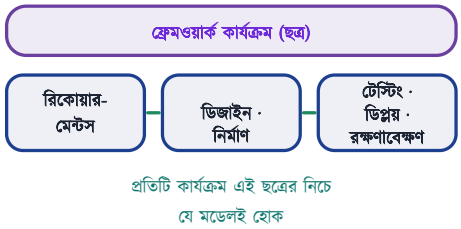
পরীক্ষার ফাঁদ: **সাধারণ ক্ষেত্রে সঠিকতা প্রমাণ করা যায় না** — টেস্টিং কেবল নিশ্চয়তা বাড়ায়, তাই "১০০% সঠিক" সার্টিফাইযোগ্য নয়।

পরীক্ষার ফাঁদ: **রক্ষণাবেক্ষণযোগ্যতা ≠ ব্যবহারযোগ্যতা** — প্রথমটি ডেভেলপারের জন্য, দ্বিতীয়টি ব্যবহারকারীর জন্য।

৩ সফটওয়্যার প্রসেস, ফ্রেমওয়ার্ক ও সর্বজনীন অনুশীলন

সফটওয়্যার প্রসেস: এমন কার্যক্রমের সেট, তার **ক্রম, ভূমিকা, ইনপুট ও আউটপুট**, যা একটি প্রয়োজনকে চলমান পণ্যে রূপান্তর করে। একটি **মডেল / প্যারাডাইম** হলো তার একটি নির্দিষ্ট *বিন্যাস* — বাস্তবে দলগুলো hybrid কাজ করে।

পরিভাষা	অর্থ
প্রসেস (process)	কার্যক্রম + ক্রম + ভূমিকা + artifact
মেথডোলজি	প্রসেস **এবং** তার নিয়ম-নীতিমালা
মডেল / প্যারাডাইম	প্রসেসের **আকৃতি**
টুল	কোনো কার্যক্রম স্বয়ংক্রিয় করে (CASE, IDE)



জেনেরিক ফ্রেমওয়ার্ক: প্রতিটি কার্যক্রমের ওপরে ছত্র

ছয়টি ফ্রেমওয়ার্ক কার্যক্রম — *সব* মডেলের ওপরেই চাপে

ফ্রেমওয়ার্ক কার্যক্রম	আসলে যা করে	আউটপুট
প্রজেক্ট পরিকল্পনা	শ্রম, খরচ ও সময়ের হিসাব, মাইলফলক ঠিক করে	প্রজেক্ট পরিকল্পনা
ট্র্যাকিং ও নিয়ন্ত্রণ	পরিকল্পনার সাপেক্ষে অগ্রগতি মাপে ও সংশোধন করে	অগ্রগতি প্রতিবেদন
ঝুঁকি বিশ্লেষণ	প্রযুক্তি, সময়, বাজেট ও জনবলের ঝুঁকি খোঁজে	ঝুঁকি রেজিস্টার
মান নিশ্চয়তা (QA)	QA পরিকল্পনা, মানদণ্ড ও আনুষ্ঠানিক পর্যালোচনা	QA পরিকল্পনা
কনফিগারেশন ম্যানেজমেন্ট	baseline, versioning ও পরিবর্তন নিয়ন্ত্রণ	baseline
টেকনিক্যাল রিভিউ	গ্রহণের আগে প্রতিটি artifact যাচাই করার দরজা	রিভিউ মিনিট

তালিকায় যোগ করো: **রিকোয়ারমেন্ট ট্রেসেবিলিটি** (রিকোয়ারমেন্ট → ডিজাইন → কোড → টেস্ট, একটি RTM ম্যাট্রিক্স), **কোডের সাথেই ইউনিট টেস্ট**, **প্রোটোটাইপ করে বোঝা**, **লাইব্রেরি ও COTS পুনর্ব্যবহার**।

মনে রাখার কৌশল: ছত্রের ক্রম: **প**-**ট্রা**-**ঝু**-**মা**-**ক**-**টি** — **প**রিকল্পনা, **ট্রা**কিং, **ঝুঁ**কি, **মা**ন নিশ্চয়তা, **ক**নফিগারেশন, **টি**কনিক্যাল রিভিউ।

পরীক্ষার ফাঁদ: **জেনেরিক প্রসেস ফ্রেমওয়ার্ক কোনো প্রসেস মডেল নয়** — এটি ছত্র। "কোনটি মডেল?" — উত্তর waterfall / spiral / V-model, কখনো "framework" নয়।

মানক ISO/IEC 12207 · পরিপক্কতা CMM / CMMI ৫ স্তর

৪ জেনেরিক প্রসেস প্যাটার্ন (১৪টি)

এগুলো কী: পুনরাবৃত্ত প্রসেস সমস্যার **নামধার্য সমাধান**। একটি লাইফ-সাইকেল মডেল আসলে এদের একটি **নির্বাচন** — কোনোটিই সর্বজনীনভাবে সেরা নয়।

প্রযুক্তি (৫) ও প্রসেস (৩) প্যাটার্ন

- **পুনর্ব্যবহারযোগ্য আর্কিটেকচারের বিবর্তন**
- **ইটারেটিভ ডিজাইন**
- **ইনক্রিমেন্টাল ডেলিভারি**
- **ইভোলিউশনারি প্রোটোটাইপিং**
- **স্পাইরালিং**
- **স্টেজড ডেলিভারি** (প্রসেস)

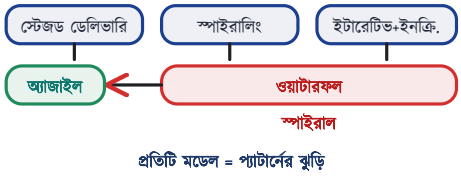
প্র্যাকটিস প্যাটার্ন (৬)

- **টুল-ভিত্তিক রূপান্তর ও অটোমেশন**
- **কম্পিউটার-সহায়ক সফটওয়্যার ইঞ্জিনিয়ারিং (CASE)**
- **ডেটা ডিজাইনের যুক্তিসংক্ষেপ**
- **ডেটা ডিজাইনের জন্য প্রোটোটাইপিং**
- **অ্যালগরিদম ও ডেটা স্ট্রাকচারে গাণিতিক চন্দ্র**
- **অবজেক্ট-ওরিয়েন্টেড ডিজাইন প্যারাডাইম**

- **ফরমাল মেথডের শৃঙ্খলা** (প্রসেস)
- **এক্সিকিউটেবল স্পেসিফিকেশন** (প্রসেস)

পরীক্ষার ফাঁদ: প্যাটার্ন কোনো বিধিবিধান নয়। "অ্যালগরিদম ও ডেটা স্ট্রাকচারের ডিজাইনে গাণিতিক ব্যবহারের শৃঙ্খলা কোন প্যাটার্ন?" → **গাণিতিক চন্দ্র**, "ফরমাল মেথড" নয়।

মডেল = প্যাটার্নের ঝুড়ি: waterfall = স্টেজড ডেলিভারি · spiral = স্পাইরালিং + ঝুঁকি · agile = ইটারেটিভ + ইনক্রিমেন্টাল + প্রোটোটাইপিং।



তিনটি প্যাটার্ন থেকে তিনটি মডেল

৫ তথ্যের প্রকার ও গুণাবলী

শ্রেণিবিভাগের ভিত্তি	প্রকার ও উদাহরণ
প্রকৃতি অনুযায়ী	**পরিমাণগত** (বিক্রয়, তাপমাত্রা) · **গুণগত** (গ্রাহক মতামত)
সময় অনুযায়ী	**রিয়েল-টাইম** (বর্তমান দাম) · **ঐতিহাসিক** (গত মাসের বিক্রয়)
উৎস অনুযায়ী	**অভ্যন্তরীণ** (নিজস্ব বিক্রয়তথ্য) · **বাহ্যিক** (বাজার গবেষণা)
উদ্দেশ্য অনুযায়ী	**কৌশলগত** (সম্প্রসারণ) · **কৌশলবৃত্তিক** (সম্পদ বরাদ্দ) · **অপারেশনাল** (দৈনিক মজুদ)

ভালো তথ্যের গুণ:

① **নির্ভুলতা** — সঠিক, ত্রুটিমুক্ত · ② **প্রাসঙ্গিকতা** — সিদ্ধান্তের জন্য

③ **সম্পূর্ণতা** — সব বিবরণ · ④ **সময়োপযোগিতা** — যখন দরকার তখন

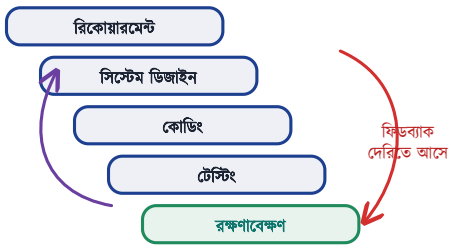
⑤ **নির্ভরযোগ্যতা** — বিশ্বস্ত উৎস · ⑥ বোধগম্য

⑦ ধারাবাহিক · ⑧ অ্যাক্সেসযোগ্য (সহজে পাওয়া যায়)

৬ প্রসেস মডেল I — ওয়াটারফল ও V-মডেল

সাধারণ ভিত্তি: সব মডেল একই কার্যক্রম করে। পার্থক্য হয় **ক্রমে**, **ফিডব্যাকে** ও **টেস্টের সময়ে**। মডেলকে ধাপের তালিকা হিসেবে মুখস্থ করবেন না — মুখস্থ করুন তার **আকৃতি**।

৬.১ ওয়াটারফল (রৈখিক) মডেল



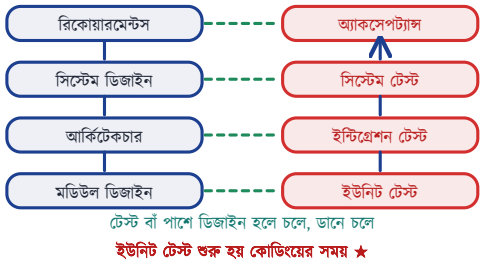
প্রতি ধাপ একবার; ফিরে যাওয়া ব্যয়বহুল

- **উৎপত্তি: বেনিংটন, IBM, ১৯৫৬; রয়েস, ১৯৭০** এটি এঁকে দুর্বলতা তুলে ধরেন।
- **আকৃতি:** ধাপগুলো কঠোর ক্রমে, প্রতিটি **স্বাক্ষরিত** হওয়ার পরেই পরেরটি শুরু।
- **যেখানে:** রিকোয়ারমেন্ট বোঝা ও চূড়ান্ত; নির্দিষ্ট দামের চুক্তি; নিয়ন্ত্রিত কাজ।
- **শক্তি:** শৃঙ্খলা, প্রতিটি দরজায় একটি নথি, সহজ বাজেটিং, **চুক্তির জন্য উপযোগী**।
- **দুর্বলতা:** দেরিতে কাজের সফটওয়্যার, ভুল রিকোয়ারমেন্ট দেরিতে ধরা পড়ে, **পরিবর্তনে অদৃঢ়**।

পরীক্ষার ফাঁদ: রয়েস ওয়াটারফলকে সফলতার সূত্র হিসেবে প্রস্তাব করেননি — তাঁর ১৯৭০ পত্রে এটি **শুরুতর দুর্বলতাসহ** একটি মডেল হিসেবে দেখানো হয়েছে, যার সমাধান হলো প্রতিটি ধাপে যাচাইয়ের ফল ফিরিয়ে দেওয়া।

৬.২ V-মডেল (যাচাই ও বৈধতা মডেল)

বাম বাহু — ডেভেলপমেন্ট	ডান বাহু — মিল রাখা টেস্ট পর্যায়
রিকোয়ারমেন্টস	**অ্যাকসেপট্যান্স** টেস্টিং (ব্যবহারকারীর দৃষ্টিভঙ্গি)
সিস্টেম ডিজাইন	**সিস্টেম** টেস্টিং (গোটা সিস্টেম)
আর্কিটেকচারাল ডিজাইন	**ইন্টিগ্রেশন** টেস্টিং (মডিউল)
মডিউল ডিজাইন	**ইউনিট** টেস্টিং (ডেভেলপার করেন)



প্রতিটি টেস্ট ডিজাইন শুরু হলেই পরিকল্পনা করা হয়

মনে রাখার কৌশল: জোড়া মনে রাখতে V-মডেল **নিচ থেকে ওপরে** পড়ো: **মডিউল→ইউনিট, আর্কিটেকচার→ইন্টিগ্রেশন, সিস্টেম→সিস্টেম, রিকোয়ারমেন্টস→অ্যাকসেপট্যান্স**।

৭ প্রসেস মডেল II — প্রোটোটাইপিং, ইনক্রিমেন্টাল, বিগ ব্যাং

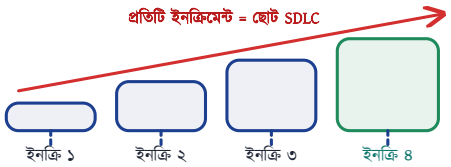
৭.১ প্রোটোটাইপিং মডেল



উপরের সারি = ইভোলিউশনারি (রাখা হয়); নিচের সারি = ছুঁড়ে ফেলা (বাদ দিয়ে নতুন করে বানানো)

- **রিকোয়ারমেন্ট অস্পষ্ট** হলে ব্যবহার করো — গড়ার আগে ব্যবহারকারীর সাথে অনুসন্ধান করে।
- **১ম জেনারেশন / থ্রো-অ্যাওয়ে:** কাগজ বা স্ক্রিনের নমুনা, শুধু রিকোয়ারমেন্ট বের করতে, তারপর **বাদ**।
- **২য় জেনারেশন / ইভোলিউশনারি:** প্রোটোটাইপ বারবার উন্নত করে চলে, একদিন সেটিই **পণ্য** হয়।
- **শক্তি:** রিকোয়ারমেন্টের অস্পষ্টতা দ্রুত শেষ; ব্যবহারকারী শীঘ্রই কাজের জিনিস দেখে।
- **দুর্বলতা:** ব্যবহারকারী প্রোটোটাইপকেই পণ্য ভাবে, তাই মানের কাঁটা বাঁচিয়ে যায়।
- **ফাঁদ:** প্রোটোটাইপিং মানে "দুইবার বানিয়ে খারাপটা রাখা" **নয়**।

৭.২ ইনক্রিমেন্টাল (বিবর্তনমূলক ডেলিভারি) মডেল



ব্যবহারযোগ্য টুকরো: আগে কোর, তারপর বাকি

- **আকৃতি:** সিস্টেম **কাজ করা, রিলিজযোগ্য ইনক্রিমেন্টে** বাড়ে; প্রতিটিতে পুরো SDLC চলে। **প্রথম ইনক্রিমেন্টই কোর পণ্য**, খালি খোলস নয়।
- **যেখানে:** রিকোয়ারমেন্ট তালিকাভুক্ত ও **অগ্রাধিকার্ভুক্ত** করা যায়; গ্রাহক আগেই ব্যবহার শুরু করতে পারে।
- **শক্তি:** আগেই ব্যবহারযোগ্য সফটওয়্যার, প্রতি রিলিজে ফিডব্যাক, ঝুঁকি ছড়িয়ে যায়।
- **দুর্বলতা:** মোট খরচ ওয়াটারফলের চেয়ে বেশি হতে পারে; পরের ইনক্রিমেন্টে **রিকোয়ারমেন্ট ও ডিজাইন** আবার বদলায়।
- **ফাঁদ: ইটারেটিভ =/ ইনক্রিমেন্টাল** — ইটারেটিভ মানে একই প্রসেস পুনরাবৃত্তি, ইনক্রিমেন্টাল মানে টুকরো যোগ।

৭.৩ বিগ ব্যাং মডেল (গড়ো, তারপর ছাড়ো — কোনো প্রসেস নেই)

- **আকৃতি: কোনো ধাপই নেই** — রিকোয়ারমেন্ট, ডিজাইন, কোড ও টেস্ট সব একসাথে; একটি মাত্র রিলিজ।
- **যেখানে:** খুব ছোট, ঝুঁকিহীন, ছুঁড়ে ফেলার মতো পরীক্ষা, যেখানে সময় পদ্ধতির চেয়ে গুরুত্বপূর্ণ।
- **শক্তি:** কোনো প্রসেস-ওভারহেড নেই, সবচেয়ে দ্রুত প্রথম বিল্ড।
- **দুর্বলতা: বিশাল দেরির ঝুঁকি** — শেষ পর্যন্ত ফিডব্যাক নেই, কোনো মধ্যবর্তী রিলিজ নেই, পরে রক্ষণাবেক্ষণ কঠিন।



সব একসাথে, তারপর একটি রিলিজ

পরীক্ষার ফাঁদ: বিগ ব্যাং "খারাপভাবে চালানো ওয়াটারফল" **নয়** — এটি প্রসেসের **অনুপস্থিতি**।

৮ প্রসেস মডেল III — স্পাইরাল (বোয়েম) ও RAD

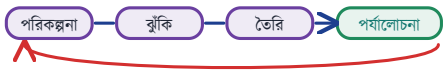


চারটি কোয়াড্রেন্ট, প্রতি লুপে একটি; পরের লুপ সস্তা

৮.১ স্পাইরাল মডেল

- **রচয়িতা: বোয়েম, ১৯৮৬**, বড় প্রকল্পে ঝুঁকি-ব্যবস্থাপনার নীতি থেকে।
- **আকৃতি: ঝুঁকি-চালিত** — প্রতি লুপে **পরিকল্পনা** → **ঝুঁকি বিশ্লেষণ** → **ইঞ্জিনিয়ারিং** → **মূল্যায়ন**; পণ্য প্রোটোটাইপ থেকে ধাপে ধাপে বড় হয়।
- **কোয়াড্রেন্ট:** ১ লক্ষ্য, বিকল্প ও সীমাবদ্ধতা · ২ বিকল্প মূল্যায়ন ও ঝুঁকি সমাধান · ৩ পরের স্তরের পণ্য তৈরি ও যাচাই · ৪ পরের পুনরাবৃত্তির পরিকল্পনা।
- **যেখানে:** বড়, ঝুঁকিপূর্ণ, ব্যর্থতা ব্যয়বহুল প্রকল্প — প্রতিরক্ষা, ব্যাংকের কোর, নিরাপত্তা-সংবেদনশীল।
- **শক্তি:** ঝুঁকিকে স্পষ্ট ও আগেই করে দেয়; **চক্রাকার** কাজের সাথে **পুনর্ব্যবহার** জুড়ে।
- **দুর্বলতা:** ব্যয়বহুল; ঝুঁকি বিশ্লেষণে দক্ষ লোক লাগে; শৃঙ্খলা না থাকলে "অফলন্ত ওয়াটারফল"।

মনে রাখার কৌশল: উপরে-বাম থেকে দড়ির দিকে ঘুরো: **ল**ক্ষ্য → **ঝ**ুঁকি → **ত**ৈরি → **প**রিকল্পনা। প্রতি লুপে একটি পূর্ণ চক্র।



একটি লুপ = পরিকল্পনা, ঝুঁকি, তৈরি, পর্যালোচনা — তারপর আবার লুপ; পরের রাউন্ড সস্তা

৮.২ RAD — র‍্যাপিড অ্যাপ্লিকেশন ডেভেলপমেন্ট

RAD: একটি **টাইম-বক্সড, ছোট দলের ও কম্পোনেন্ট-পুনর্ব্যবহারভিত্তিক** পদ্ধতি: বিশ্লেষণ, ডিজাইন ও প্রোটোটাইপিং **একই বৃত্তে ফুশে** যায় এবং পণ্য প্রায় **৯০ দিনে** তৈরি হয়। ১৯৮০-র দশকে **জেমস মার্টিন** এটি জনপ্রিয় করেন।

- **যেখানে:** ছোট দল, GUI/ব্যবসায়িক অ্যাপ, দুই পক্ষেরই রিকোয়ারমেন্ট পরিষ্কার।
- **যেখানে খাপ খায় না:** বড় মিশন-ক্রিটিক্যাল ব্যাক-এন্ড সিস্টেম, ভারী নিরাপত্তা-নির্ভরতা, বিতর্কিত দল।
- **শক্তি:** অত্যন্ত ছোট চক্র; ব্যবহারকারী সিস্টেমের সাথেই কাজ করে।
- **দুর্বলতা:** প্রযোজ্যতা সীমিত; সময়ের চাপে টেস্ট ও নথি বাইরে চলে যায়।

মনে রাখার কৌশল: RAD-এর তিনটি R: **R**apid, **R**equirements আগেই পরিষ্কার, **R**euse। প্রোটোটাইপিং রিকোয়ারমেন্ট পরিষ্কার করার পরেই RAD বসে।

অ্যাজাইল = ইটারেটিভ + ইনক্রিমেন্টাল ডেলিভারি + ইভোলিউশনারি প্রোটোটাইপিং, গ্রাহকের অংশগ্রহণসহ (অ্যাজাইল ম্যানিফেস্টো, ২০০১)।

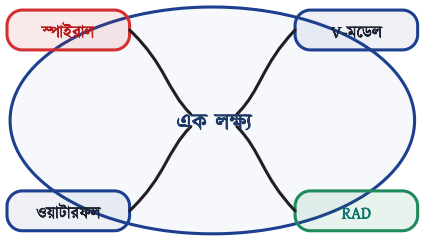
৯ সাত মডেল, এক টেবিল: আকৃতি · উপযুক্ততা · শক্তি · দুর্বলতা

মডেল	আকৃতি	কখন উপযোগী	শক্তি	দুর্বলতা
ওয়াটারফল	একবার, রৈখিক	রিকোয়ারমেন্ট জানা ও চূড়ান্ত	নথিযুক্ত, বাজেটযোগ্য	অদৃঢ়; দেরির বাগ ×১০০
V-মডেল	ডেভ বাঁ, টেস্ট ডান	নিরাপত্তা-সংবেদনশীল, পরীক্ষাযোগ্য	টেস্ট আগেই ডিজাইন হয়	ব্যয়বহুল, শুরুতেই ভারী
প্রোটোটাইপিং	বিশ্লেষণ → প্রোটোটাইপ → বিল্ড	রিকোয়ারমেন্ট অস্পষ্ট, UI-নির্ভর	অস্পষ্টতা দ্রুত শেষ	প্রোটোটাইপকেই পণ্য ভাবা
ইনক্রিমেন্টাল	রিলিজের সিঁড়ি	রিকোয়ারমেন্ট অগ্রাধিকার্ভুক্ত	আগেই ব্যবহারযোগ্য, ঝুঁকি ছড়ায়	ওয়াটারফলের চেয়ে দামি
স্পাইরাল	চার-কোয়াড্রেন্ট ঝুঁকি লুপ	বড়, ঝুঁকিপূর্ণ প্রকল্প	স্পষ্ট ঝুঁকি বিশ্লেষণ	ঝুঁকি-বিশ্লেষক লাগে
RAD	বিশ্লেষণ+ডিজাইন+বিল্ড একসাথে	ছোট দল, GUI অ্যাপ, রিকোয়ারমেন্ট জানা	খুব ছোট চক্র	সীমিত প্রযোজ্যতা
বিগ ব্যাং	কোনো ধাপই নেই	ছোট, ঝুঁকিহীন, ছুঁড়ে ফেলার মতো	প্রসেস-ওভারহেড নেই	ফিডব্যাক নেই, ঝুঁকি বিশাল

মনে রাখার কৌশল: মডেল বাছাই: **১. রিকোয়ারমেন্ট কি পরিষ্কার?** না → **প্রোটোটাইপিং**; হ্যাঁ → **২. ঝুঁকি কি বেশি?** হ্যাঁ → **স্পাইরাল** বা **V-মডেল**; না → **৩. ছোট GUI অ্যাপ?** হ্যাঁ → **RAD**; নইলে **ইনক্রিমেন্টাল**।

পরীক্ষার ফাঁদ: "আগেই কাজের পণ্য?" → ইনক্রিমেন্টাল / অ্যাজাইল / RAD · "প্রতিটি ডিজাইনের সাথে টেস্ট পর্যায়?" → V-মডেল · "ঝুঁকি-চালিত?" → স্পাইরাল। **ইটারেটিভ** ও **ইনক্রিমেন্টাল** কখনো গুলিয়ে ফেলো না।

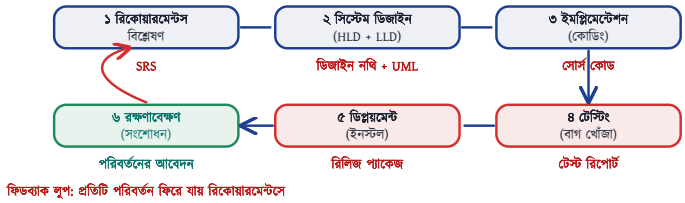
পারস্পরিক নির্দেশ: COCOMO হিসাব, Gantt / PERT / CPM শিডিউলিং ও ঝুঁকি-ব্যবস্থাপনা **প্রজেক্ট ম্যানেজমেন্ট, মেট্রিক, ঝুঁকি ও শিডিউলিং** লেসনে।



প্রতিটি মডেল একই কার্যক্রম, ভিন্ন বিন্যাসে — সঠিক মডেল, সঠিক সময়ে

১০ SDLC ধাপ: প্রতিটির কার্যক্রম ও ডেলিভারেবল

SDLC: Software Development Life Cycle — সম্ভাব্যতা যাচাই থেকে অবসর পর্যন্ত ধাপগুলোর ক্রম। লিখিত উত্তরে প্রতিটি ধাপের **প্রধান কার্যক্রম** ও **উৎপন্ন artifact** লিখতে হয়।



ধাপের সাথে ডেলিভারেবল (লাল) ও ফিরে যাওয়া ফিডব্যাক লুপ

ধাপ	প্রধান কার্যক্রম	ডেলিভারেবল (artifact)
০ সম্ভাব্যতা	TELOS যাচাই — প্রযুক্তিগত, অর্থনৈতিক, আইনগত ও সময়গতভাবে সম্ভব কিনা?	**ফিজিবিলিটি রিপোর্ট**, প্রজেক্ট চার্টার
১ রিকোয়ারমেন্টস বিশ্লেষণ ও স্পেসিফিকেশন	তথ্য সংগ্রহ (সাক্ষাৎকার, প্রশ্নোত্তর, পর্যবেক্ষণ, প্রোটোটাইপ), বিশ্লেষণ, লেখা, যাচাই, পরিবর্তন ব্যবস্থাপনা	SRS ডকুমেন্ট, ইউজ কেস, ব্যবহারকারী ও সিস্টেম রিকোয়ারমেন্ট
২ সিস্টেম ডিজাইন	আর্কিটেকচার, মডিউল, ইন্টারফেস, ডেটা ডিজাইন; HLD তারপর LLD; UML ডায়াগ্রাম	ডিজাইন ডকুমেন্ট (HLD/LLD), UML ক্লাস ও সিকোয়েন্স ডায়াগ্রাম
৩ ইমপ্লিমেন্টেশন / কোডিং	কোড লেখা, লেখার সাথে ইউনিট টেস্ট, কোড রিভিউ, মডিউল একীভূতকরণ	সোর্স কোড, ইউনিট-টেস্ট ফলাফল, বিল্ড
৪ টেস্টিং	যাচাই ও বৈধতা; ইউনিট → ইন্টিগ্রেশন → সিস্টেম → অ্যাকসেপট্যান্স	টেস্ট পরিকল্পনা ও কেস, ডিফেক্ট লগ, টেস্ট রিপোর্ট
৫ ডিপ্লয়মেন্ট	ইনস্টল, ডেটা মাইগ্রেশন, ব্যবহারকারী প্রশিক্ষণ, হস্তান্তর, পরে-ডিপ্লয় পর্যালোচনা	রিলিজ প্যাকেজ, ইনস্টল ও ব্যবহার-নির্দেশিকা
৬ রক্ষণাবেক্ষণ	সংশোধন (বাগ), অ্যাডাপ্টিভ (নতুন প্ল্যাটফর্ম), পারফেক্টিভ (নতুন ফিচার), প্রিভেন্টিভ (রিফ্যাক্টরিং)	পরিবর্তনের আবেদন, হালনাগাদ রিলিজ, রক্ষণাবেক্ষণ রিপোর্ট

মনে রাখার কৌশল: রক্ষণাবেক্ষণের চার প্রকার: **স**ংশোধন = বাগ · **অ্যা**ডাপ্টিভ = নতুন OS · **পে**রফেক্টিভ = নতুন ফিচার · **প্রি**ভেন্টিভ = ভবিষ্যতের বাগ থামানো। মোট জীবনচক্র খরচের প্রায় **৬০–৮০%** এখানেই।

মনে রাখার কৌশল: দেরিতে ধরার খরচ: বাগ যত দেরিতে ধরা পড়ে, সংশোধন তত দামি: **×১** → **×৫** → **×১০** → **×১৫–৫০** → **×১০০**।

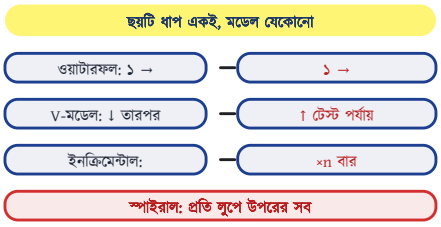
১১ প্রতিটি মডেল ধাপগুলোকে কোথায় রাখে?

ভাবনা: কোনো মডেল কোনো ধাপ মুছে ফেলে না — সে ধাপগুলোকে **ক্রম বদলে, পুনরাবৃত্তি করে, ওভারল্যাপ করে বা একীভূত করে**। এই এক বাক্যেই "মডেল তুলনা" লিখিত প্রশ্নের বেশিরভাগ উত্তর।

মডেল	ধাপ পুনরাবৃত্তি?	টেস্টিং কখন শুরু	ফিডব্যাক কোথা থেকে	কোন ধাপে জোর
ওয়াটারফল	না, একবার	কোডিং শেষে	পরের ধাপ থেকে, দেরিতে	নথি
V-মডেল	না, ৪টি টেস্ট পর্যায়	**পরিকল্পনা** প্রতিটি ডিজাইনের সাথে	মিল রাখা টেস্ট পর্যায়	যাচাই
প্রোটোটাইপিং	রিকোয়ারমেন্টস দুইবার	প্রোটোটাইপের ওপরেই	ব্যবহারকারীর প্রতিক্রিয়া	রিকোয়ারমেন্টস
ইনক্রিমেন্টাল	হ্যাঁ — **প্রতি ইনক্রিমেন্টে পুরো** SDLC	প্রতিটি ইনক্রিমেন্টে	প্রতিটি রিলিজের পর	ডেলিভারি
স্পাইরাল	হ্যাঁ — প্রতি লুপে	প্রতি লুপে	প্রতি লুপের মূল্যায়ন	ঝুঁকি বিশ্লেষণ
RAD	এক বাক্সে মিশ্রিত	৯০ দিন ধরে নিয়ন্ত্রিত	ক্রমাগত, আনুষ্ঠানিক	গতি, পুনর্ব্যবহার
বিগ ব্যাং	কোনো ধাপই নেই	সব তৈরির পর	একমাত্র রিলিজে	নেই

মনে রাখার কৌশল: প্রতিটি মডেল একটি প্রশ্নের উত্তর দেয়। **ওয়াটারফল:** "শেষ হয়েছে?" · **V-মডেল:** "পরীক্ষা হয়েছে?" · **প্রোটোটাইপিং:** "সঠিক জিনিস তো?" · **ইনক্রিমেন্টাল:** "কিছুটা বের হয়েছে?" · **স্পাইরাল:** "ঝুঁকি নিরাপদ?" · **RAD:** "দ্রুত হয়েছে?"

পরীক্ষার ফাঁদ: "কোন মডেলে টেস্টিং সবার আগে শুরু হয়?" — বিকল্পে "ইউনিটের ডিজাইনের সময়" থাকলে উত্তর **V-মডেল**। "কোন মডেলে কোনো ধাপ নেই?" → **বিগ ব্যাং**; "পুরো জীবনচক্র কোন মডেলে পুনরাবৃত্তি হয়?" → **ইনক্রিমেন্টাল / ইটারেটিভ / স্পাইরাল**, ওয়াটারফল কখনো নয়।



ধাপ কখনো বদলায় না — শুধু তার বিন্যাস বদলায়

সিকোয়েন্সিয়াল = নির্দিষ্ট ক্রম · ইটারেটিভ = প্রসেস পুনরাবৃত্তি · ইনক্রিমেন্টাল = টুকরো যোগ।

পরীক্ষার আগের রাতের পাতা

ফ্যাক্ট শিট:

SE = পদ্ধতিগত, শৃঙ্খলাবদ্ধ, পরিমাপযোগ্য

Coined — **NATO, গার্মিশ, ১৯৬৮**

বৈশিষ্ট্য: জটিলতা, অনুবর্তিতা, পরিবর্তনশীলতা, অদৃশ্যতা

লক্ষ্য: ভালো পণ্য · পুনরাবৃত্ত প্রসেস · সময় ও বাজেটে

গুণ: সঠিকতা, নির্ভরযোগ্যতা, দক্ষতা, রক্ষণাবেক্ষণযোগ্যতা, পোর্টেবিলিটি, ব্যবহারযোগ্যতা

\+ নিরাপত্তা, নমনীয়তা, পরীক্ষাযোগ্যতা, পুনর্ব্যবহারযোগ্যতা

ফ্রেমওয়ার্ক: পরিকল্পনা, ট্র্যাকিং, ঝুঁকি, QA, কনফিগারেশন, রিভিউ

ISO/IEC 12207 · CMM/CMMI ৫ স্তর

প্যাটার্ন: ৫ প্রযুক্তি + ৩ প্রসেস + ৬ প্র্যাকটিস = **১৪**

ওয়াটারফল: বেনিংটন ১৯৫৬ · রয়েস ১৯৭০

V-মডেল: রিকোয়া→অ্যাকসেপট্যান্স, সিস্টেম→সিস্টেম, আর্কি→ইন্টিগ্রেশন, মডিউল→ইউনিট

স্পাইরাল: বোয়েম ১৯৮৬ · ৪ কোয়াড্রেন্ট · ঝুঁকি-চালিত

RAD: ~৯০ দিন, ছোট দল, GUI, রিকোয়ারমেন্ট জানা

বিগ ব্যাং: কোনো ধাপই নেই

SDLC: ফিজিবিলিটি, রিকোয়া, ডিজাইন, কোড, টেস্ট, ডিপ্লয়, রক্ষণাবেক্ষণ

SRS = রিকোয়ারমেন্টস-ফেজের ডেলিভারেবল

রক্ষণাবেক্ষণ: সংশোধন, অ্যাডাপ্টিভ, পারফেক্টিভ, প্রিভেন্টিভ

বাগ ঠিক করার খরচ: ×১ → ×৫ → ×১০ → ×১৫–৫০ → ×১০০

তথ্য: পরিমাণ/গুণ, রিয়েল-টাইম/ঐতিহাসিক, অভ্যন্তরীণ/বাহ্যিক

ভালো তথ্য: নির্ভুল, প্রাসঙ্গিক, সম্পূর্ণ, সময়োপযোগী, নির্ভরযোগ্য

MCQ-র জন্য সেরা ১২টি এক-লাইনের তথ্য

1. **SE** coined ১৯৬৮-এ NATO সম্মেলনে
2. ওয়াটারফল = **রৈখিক**, একবার
3. V-মডেল প্রতিটি ধাপের সাথে **টেস্ট পর্যায়**
7. প্রোটোটাইপিং উপযোগী **অস্পষ্ট** রিকোয়ারমেন্টে
8. ইনক্রিমেন্টাল = **কাজের টুকরো**
9. RAD-এর রিকোয়ারমেন্ট **জানা** থাকে

4. স্পাইরাল = **ঝুঁকি-চালিত** (বোয়েম, ১৯৮৬)
5. বিগ ব্যাং = **কোনো ধাপ নেই**
6. SRS লেখা হয় **রিকোয়ারমেন্টস** ধাপে
10. রক্ষণাবেক্ষণ = **সবচেয়ে বড়** খরচ
11. QA = **রিভিউ + মানদণ্ড**
12. অ্যাজাইল ম্যানিফেস্টো = **২০০১**

সবচেয়ে বেশি ফাঁদ: রয়েস ওয়াটারফলকে সফলতার সূত্র হিসেবে দেননি · প্রোটোটাইপিং =/ দুইবার কোড লেখা · ইটারেটিভ =/ ইনক্রিমেন্টাল · টেস্টিং শুধু কোডিংয়ের পরে নয় (V-মডেল) · রক্ষণাবেক্ষণ শেষ নয় — পরিবর্তন রিকোয়ারমেন্টসে ফিরে যায় · সাধারণ ক্ষেত্রে সঠিকতা প্রমাণযোগ্য নয়।

সব কৌশল একসাথে: জ-অ-পে-অ = সফটওয়্যারের বৈশিষ্ট্য · স-অ্যা-পে-প্রি = রক্ষণাবেক্ষণের চার প্রকার · V-মডেল নিচ থেকে ওপরে পড়ো · স্পাইরাল দড়ির দিকে ঘোরো: লক্ষ্য, ঝুঁকি, তৈরি, পরিকল্পনা · তিন প্রশ্নে (পরিষ্কার? ঝুঁকিপূর্ণ? GUI?) মডেল বাছাও।



ছয়টি ধাপ কখনো বদলায় না — শুধু মডেল বদলায়

✓ অনুশীলন MCQ (পরীক্ষার ধাঁচে, ১০ নম্বর)

১. কোন মডেল ঝুঁকি-চালিত, প্রতি লুপে পরিকল্পনা→ঝুঁকি→তৈরি করে?
ক) ওয়াটারফল মডেল খ) V-মডেল
গ) স্পাইরাল মডেল ঘ) বিগ ব্যাং মডেল

২. SRS ডকুমেন্ট কোন ধাপে তৈরি হয়?
ক) বিশ্লেষণ ও স্পেসি. খ) ডিজাইন
গ) কোডিং ঘ) রক্ষণাবেক্ষণ

৩. কোন মডেলে প্রতিটি ডেভ ধাপের সাথে একটি টেস্ট ধাপ জোড়া হয়?
ক) ইনক্রিমেন্টাল খ) V-মডেল
গ) প্রোটোটাইপিং ঘ) RAD মডেল

৪. প্রোটোটাইপিং মডেল কখন উপযোগী?
ক) রিকোয়ারমেন্ট জানা ও স্থির খ) ছোট, পরিচিত পরিবর্তন
গ) গ্রাহকের যোগাযোগ নেই ঘ) রিকোয়ারমেন্ট অস্পষ্ট

৫. "Software Engineering" শব্দটি ১৯৬৮ সালে কোথায়?
ক) বেঙ্গালুরু খ) গার্মিশ, জার্মানি (NATO)
গ) ক্যামব্রিজ ঘ) টোকিও

৬. কোন মডেলে কোনো আনুষ্ঠানিক প্রসেসই নেই?
ক) ওয়াটারফল খ) V-মডেল
গ) ইনক্রিমেন্টাল ঘ) বিগ ব্যাং

৭. স্পাইরাল মডেল কে প্রস্তাব করেন?
ক) B. W. বোয়েম খ) জে. মার্টিন
গ) কে. বেক ঘ) ডব্লিউ. রয়েস

৮. RAD কোন কাজের জন্য উপযোগী?
ক) এয়ার-ট্রাফিক কন্ট্রোল খ) ছোট GUI অ্যাপ
গ) নতুন কম্পাইলার ঘ) অপারেটিং সিস্টেম কার্নেল

৯. মোট জীবনচক্র খরচের প্রায় কত অংশ রক্ষণাবেক্ষণে?
ক) ৫% খ) ২০%
গ) ৭০% ঘ) ১০০%

১০. প্রোডাকশনে ঠিক করা বাগ, রিকোয়ারমেন্টের বাগের কত গুণ দামি?
ক) ৫ খ) ১০
গ) ১৫ ঘ) ১০০

উত্তরমালা → ১-গ · ২-ক · ৩-খ · ৪-ঘ · ৫-খ · ৬-ঘ · ৭-ক · ৮-খ · ৯-গ · ১০-ঘ

লিখিত প্রশ্ন: একটি সংস্থাকে তৈরি করতে হবে (i) আইনে নির্ধারিত বেতনের সিস্টেম, (ii) নাগরিক পোর্টাল যার প্রয়োজন অজানা, এবং (iii) একটি পেমেন্ট গেটওয়ে যেখানে নিরাপত্তা-ব্যর্থতা ব্যয়বহুল। প্রতিটির জন্য মডেলের নাম ও যুক্তি দাও, সাথে আকৃতি, একটি শক্তি ও একটি দুর্বলতা। ১০ নম্বর

১ **(i) বেতন → ওয়াটারফল + V-মডেল টেস্ট।** রৈখিক ও স্বাক্ষরিত; বাজেটযোগ্য, তবে আইন বদলালে নিচের সব কাজ নতুন করে হয়। ৪

২ (ii) **পোর্টাল** → **প্রোটোটাইপিং, ইনক্রিমেন্টাল**। রিকোয়ারমেন্টের অস্পষ্টতা আগে কেটে যায়। ৩

৩ (iii) **গেটওয়ে** → **স্পাইরাল**। ঝুঁকি আগেই চিহ্নিত ও সমাধান হয়; ব্যয়বহুল। ৩

পরীক্ষার আগে যাচাই ✓

- ✓ IEEE সংজ্ঞা ও ১৯৬৮-এর উৎপত্তি বলতে পারি
- ✓ ৭টি মডেলের নাম ও আকৃতি আঁকতে পারি
- ✓ V-মডেলের জোড়া মডিউল → ইউনিট থেকে বলতে পারি
- ✓ ৬টি ধাপ ও প্রতিটির ডেলিভারেবল বলতে পারি

আরও অনুশীলন (লিখিত): ১. চার কোয়াড্রেন্টসহ স্পাইরাল মডেল আঁকো। ২. অস্পষ্ট রিকোয়ারমেন্টে ওয়াটারফল কেন ব্যর্থ। ৩. ইটারেটিভ বনাম ইনক্রিমেন্টাল।